

Project Progress Report: Week 5 & Week 6

Project Title: CodeDNA – Biomorphic Code Visualization Engine

Task Domain: Advanced Parsing & Creature Mapping Logic

1. Advanced Parsing (Week 5)

Week 5 extended the ASTParser class built in Week 4 to move beyond basic element counting into structural analysis. The core addition is accurate nested-loop detection, alongside comment extraction, additional structural metrics, and a scoring system that turns raw counts into complexity and health values.

1.1 Nested Loop Detection

Nested loops are detected by tracking loop depth as the file is read line by line. A running counter (`currentLoopDepth`) is incremented each time a `for`, `while`, or `do` is matched, and decremented when a closing brace is encountered. The maximum depth reached during the scan (`loopDepthMax`) determines the nested loop count:

```
nestedLoopCount = max(0, loopDepthMax - 1)
```

This means a single loop contributes 0 to the nested count, while a loop inside a loop (depth 2) contributes 1, correctly distinguishing simple iteration from nested structural complexity.

1.2 Comment Extraction

The parser now classifies every line as blank, comment, or code. It tracks an `inBlockComment` flag so that multi-line `/* */` and `/** */` blocks are fully counted, in addition to single-line `//` comments. The resulting comment ratio feeds directly into the health score calculation described below.

1.3 Additional Metrics

Beyond the Week 4 counts, the parser now also tracks:

- Try/catch block count (`tryCount`)
- Import statement count (`importCount`)
- Per-method line counts (`methodLineCounts`), used later to size tentacles
- Top keyword frequency, filtering out common Java reserved words to surface meaningful identifiers

1.4 Complexity & Health Scoring

Two derived scores summarize each file:

- Complexity Score (0–100): a weighted sum $\text{methods} \times 3$ (capped 30), $\text{loops} \times 4$ (capped 20), $\text{ifs} \times 2$ (capped 20), $\text{nested loops} \times 10$ (capped 15), $\text{classes} \times 5$ (capped 10), $\text{try blocks} \times 2$ (capped 5).
- Health Score (0–100): starts at a baseline of 60, adds up to 30 points for comment ratio, subtracts 8 points per nested loop, and subtracts 2 points for each method beyond 10, clamped between 5 and 100.

These two scores are what Week 6's CreatureGenerator uses to decide the creature's color, personality, and pulse animation speed.

2. Creature Mapping Logic (Week 6)

Week 6 implemented the CreatureGenerator class, which converts a CodeMetrics object into a fully-specified Creature object the biological data model that the visualization stage (SVGBuilder) will render. No drawing happens in this class; it only decides what the organism should look like.

2.1 Mapping Table

Code Element	Mapped Creature Feature
Method count	Number of tentacles (max 16)
Method line count	Tentacle length & thickness; color (red >40 lines, yellow >20, else aura color)
Loop count	Cyan orbiting cells
If count	Purple orbiting cells
Nested loop count	Red pulsing mutations near the body
Class count	Nucleus radius; gold nucleus if more than 3 classes
Health score	Body color / aura color / personality (Elegant, Warrior, Chaotic)
Complexity score	Pulse animation speed (higher complexity = faster pulse)

2.2 Deterministic Randomization

Each creature uses a Random instance seeded from the input file's name (`new Random(m.fileName.hashCode())`). This keeps organic variation in cell placement and wave timing while guaranteeing the same input file always produces the same creature — reinforcing the project's core idea that the organism is a direct, repeatable translation of the code's DNA rather than a random image.

2.3 Body, Aura, and Personality

Body radius scales with code line count (clamped between 55 and 120). The health score is split into three bands that each set a full visual identity — body color, glow color, aura color, a personality label, and a short description:

- Health ≥ 75 — blue/cyan palette, personality "Elegant", described as well-structured and clean
- Health 45–74 — green/yellow palette, personality "Warrior", described as resilient but scarred
- Health < 45 — red/orange palette, personality "Chaotic", described as mutating under stress

Pulse animation speed is derived from the complexity score, so more complex files produce a faster, more erratic-feeling pulse, clamped between 1.5 and 4.0 seconds.

2.4 Tentacles, Cells, and Mutations

Tentacles are generated one per method (capped at 16), spaced evenly around the body by angle. Each tentacle's length and thickness come from that method's line count, and its color escalates from the creature's aura color to yellow to red as the method grows longer visually flagging long methods.

Orbiting cells are generated one per loop or if statement (capped at 20): loop cells render cyan, conditional cells render purple, each placed at a randomized orbit radius and speed for organic motion.

Mutations are generated one per nested loop, rendered as small red pulsing growths near the body center directly visualizing the code smell of deep nesting.

2.5 Trait System

In addition to the visual mapping, CreatureGenerator assigns descriptive trait tags based on threshold checks — for example "Multi-functional" for more than 8 methods, "Well-documented" when comment lines exceed a third of code lines, "△ Deeply nested" for more than 2 nested loops, and "💎 Clean DNA" for a health score above 80. Files that trigger no traits default to "Simple organism". This gives each creature a short, human-readable summary alongside its visual form.

3. OOP Concepts Applied

- Encapsulation — CodeMetrics, Creature, Tentacle, Cell, and Mutation are self-contained data classes holding their own fields
- Abstraction — CreatureGenerator.generate() takes metrics in and returns a complete creature out, hiding all internal mapping logic from the caller
- Separation of Concern — ASTParser only parses; CreatureGenerator only maps data to visual attributes; rendering is left entirely to SVGBuilder
- Collections — List and Map (including LinkedHashMap for ordered method line counts) are used throughout to store parsed and generated data

- Static Methods — `parse()` and `generate()` remain static utility entry points, consistent with the Week 3–4 design

4. Development Summary

By the end of Week 6, the CodeDNA project has a fully functional data pipeline from raw Java source to a complete creature specification. The ASTParser reliably detects nested structures and computes complexity/health scores, and the CreatureGenerator translates every one of those metrics into a concrete visual attribute — tentacles, cells, mutations, body color, and personality. This closes the loop originally scoped in the Week 1 concept: every code element now has a defined, deterministic biological counterpart. The project is on track to proceed into Week 7, which focuses on rendering this creature data into an animated SVG organism.